

## Project summary (one-line)

A reproducible pipeline that scrapes drug reviews, stores raw + processed data, runs ETL via Airflow, preprocesses text (SpaCy/NLTK), trains a FastText / TextCNN sentiment model, version-controls models with MLflow, and exposes an interactive Streamlit dashboard. Containerized with Docker for reproducible local / POC deployment.

---

## 1. Final Tech Stack (Intermediate — chosen)

- Ingestion: **Python + Scrappy** (or Pandas for CSV/UCI)
- Storage: **PostgreSQL** (free cloud/dev: ElephantSQL/Render) + optional **minio** (S3-compatible) for artifacts
- Orchestration: **Apache Airflow** (Docker-based local / prod-ready)
- Preprocessing: **SpaCy + NLTK**
- Modeling: **FastText** (baseline) → **TextCNN** (improved). Option: **DistilBERT** if GPU available.
- Model tracking: **MLflow** (open-source)
- Serving & Dashboard: **Streamlit** for dashboard + a simple Flask/FastAPI model endpoint for online inference
- Containerization: **Docker + docker-compose**
- CI/CD: **GitHub Actions** (or GitLab CI)
- Monitoring: **Prometheus + Grafana** (optional) or structured logs + MLflow + simple health checks
- Secrets & config: environment variables + .env + secrets manager in prod (Vault, cloud secrets)

Why: Balanced cost, industry relevance (Airflow/Postgres/Docker), and simple to scale when needed.

---

## 2. Full Project Folder Structure (recommended)

drug-review-sentiment/

├— README.md

```
└─ docker-compose.yml
└─ .env
└─ requirements.txt
└─ Makefile
└─ infra/
|   └─ airflow/          # Airflow docker config / plugins
|   |   └─ dags/
|   |   └─ plugins/
|   |   └─ Dockerfile
|   └─ postgres/
|       └─ init.sql
|   └─ mlflow/          # mlflow server config (optional)
|       └─ docker-compose-mlflow.yml
└─ data/                # local data (gitignored large files)
|   └─ raw/
|   └─ processed/
└─ src/
|   └─ ingestion/
|   |   └─ scrapy_project/    # Scrapy spider(s)
|   |   └─ fetch_api.py      # alternate ingestion from APIs
|   |   └─ runner.py
|   └─ db/
|       └─ schema.sql        # table schemas
|       └─ db_client.py      # SQLAlchemy/psycopg2 wrapper
|       └─ migrations/      # alembic if used
```

- | └─ etl/
- | | └─ etl\_tasks.py      # transforms used by Airflow
- | |   └─ preprocess.py      # text cleaning functions
- | └─ nlp/
- | | └─ spacy\_pipeline.py
- | | └─ tokenizer.py
- | |   └─ utils.py
- | └─ models/
- | | └─ train\_fasttext.py
- | | └─ train\_textcnn.py
- | | └─ evaluate.py
- | |   └─ predict.py      # batch / online inference
- | └─ serving/
- | | └─ app.py      # FastAPI/Flask model endpoint
- | |   └─ Dockerfile
- |   └─ dashboard/
- |    └─ app\_streamlit.py
- |    └─ requirements.txt
- | └─ airflow\_dags/
- |   └─ drug\_reviews\_dag.py
- | └─ tests/
- |   └─ unit/
- |    └─ integration/
- | └─ notebooks/
- |   └─ eda.ipynb

```
| └─ model_experiments.ipynb
└─ docs/
    ├── HLD.md
    ├── LLD.md
    └─ compliance.md
```

Note: data/ and infra/postgres/init.sql should be .gitignored according to your privacy policy.

---

### 3. Database schema (Postgres) — core tables

**raw\_reviews** — raw scraped rows

```
CREATE TABLE raw_reviews (
  id SERIAL PRIMARY KEY,
  source VARCHAR(100),      -- e.g., drugs.com
  source_review_id VARCHAR(200), -- original id on website
  drug_name VARCHAR(200),
  product_name VARCHAR(255),
  user_name VARCHAR(200),
  rating INT,
  review_text TEXT,
  review_date TIMESTAMP,
  scraped_at TIMESTAMP DEFAULT now(),
  metadata JSONB           -- extra raw fields
);
```

**processed\_reviews** — cleaned + NLP columns

```
CREATE TABLE processed_reviews (
  id SERIAL PRIMARY KEY,
```

```
raw_id INT REFERENCES raw_reviews(id),
cleaned_text TEXT,
tokens TEXT[],          -- optional
lemmatized_text TEXT,
drug_normalized VARCHAR(200),
sentiment_label VARCHAR(20), -- 'positive' / 'negative' / 'neutral'
sentiment_score FLOAT,    -- model score/confidence
model_version VARCHAR(50),
features JSONB,           -- vectorized features or metadata
processed_at TIMESTAMP DEFAULT now()
);
```

**model\_registry** — (optional if not using MLflow)

```
CREATE TABLE model_registry (
  id SERIAL PRIMARY KEY,
  model_name VARCHAR(100),
  version VARCHAR(50),
  metrics JSONB,
  location VARCHAR(255), -- S3/minio or filesystem path
  created_at TIMESTAMP DEFAULT now()
);
```

---

#### 4. Airflow DAG (High-level tasks)

File: airflow\_dags/drug\_reviews\_dag.py

DAG schedule: daily / or hourly for scraping incremental pages.

Tasks:

1. start (DummyOperator)

2. `scrape_reviews` → runs Scrapy spider / or `fetch_api.py`
  - Saves to `raw_reviews` (upsert by `source_review_id`)
3. `validate_raw` → basic checks (nulls, duplicates)
4. `clean_and_preprocess` → calls `etl/preprocess.py` to create `processed_reviews` candidate table
5. `run_ner_normalize_drug_names` → uses SpaCy + custom drug lexicon to normalize drug names
6. `vectorize_features` → TF or FastText vectorization, persist features (or store pointers to feature files)
7. `train_or_update_model` (scheduled weekly or manual) → trains FastText/TextCNN on labeled data and logs to MLflow
8. `batch_infer_new_reviews` → run inference on newly processed reviews and update sentiment columns
9. `update_dashboard_data` → materialize aggregated tables for Streamlit
10. end

Airflow retries & SLA: tasks have `retry=2`, `retry_delay=10min`.

---

## 5. ETL / Preprocessing LLD (detailed steps)

### a. Scraper (Scrapy)

- Crawl pages, parse review fields to JSON.
- Respect `robots.txt` & rate limits. Log everything.
- Output: JSONL files + bulk insert into `raw_reviews`.

### b. Validation & Dedup

- Use `source_review_id` + `source` to deduplicate.
- Schema validation: reject rows missing `review_text`.

### c. Cleaning steps (ETL transform) (`etl/preprocess.py`)

1. Lowercase, strip HTML, normalize unicode.

2. Replace dosage patterns (e.g., 10 mg, 5ml) with <DOSAGE> token if desired.
3. Remove PII (usernames, emails, phone numbers) — regex + SpaCy pattern-based redaction.
4. Expand contractions (e.g., don't -> do not).
5. Remove excessive punctuation, keep emoji (optional) for sentiment signals.
6. Tokenize (SpaCy), remove stopwords, lemmatize.
7. Drug name normalization: map brand names → generic (use lookup table / fuzzy match).
8. Save cleaned\_text, lemmatized\_text, tokens into processed\_reviews.

#### **d. Feature extraction**

- Baseline: FastText embeddings (avg of word vectors) persisted as numpy arrays in minio or local artifacts/ and pointer saved in features JSON.
  - Alternate: TF-IDF sparse vector with scikit-learn saved with joblib.
- 

## **6. Model Training LLD**

### **Baseline (FastText)**

- Input: cleaned\_text + labels (if labeled). If unlabeled, use unsupervised FastText embeddings then a classifier.
- Architecture: FastText classification with n-grams (simple, fast).
- Hyperparams: lr=0.1, epoch=5-25, wordNgrams=2, dim=100.
- Metrics: Accuracy, Precision, Recall, F1 (per-class), ROC-AUC if binary.

### **Improved (TextCNN)**

- Input: tokenized text → embedding layer (pretrained FastText or trainable embedding) → conv layers (filter sizes 3/4/5) → max-pool → dense → softmax.
- Use early stopping, learning rate scheduling.
- GPU recommended but small models can train on CPU with patience.

### **Model versioning**

- Use **MLflow** to log:
  - params, metrics, artifacts (model binary, tokenizer, label map), model\_version tag
- Save model artifact path (minio or /artifacts/models/<version>/)

### **Evaluation & test-split**

- Use stratified train/val/test split.
- Save evaluation report JSON to model\_registry / MLflow.

### **Model selection**

- Train both FastText and TextCNN; select by validation F1 or AUC. Keep both in registry for A/B tests.
- 

## **7. Serving LLD (online & batch)**

### **Batch inference** (Airflow task):

- Pull latest processed\_reviews where sentiment\_label is NULL.
- Load model from MLflow artifact store.
- Run inference, update processed\_reviews.sentiment\_label, sentiment\_score, model\_version.

### **Online inference** (optional):

- Deploy small FastAPI service wrapping model predict; containerize; use gunicorn + uvicorn.
- Endpoint: /predict accepts JSON { "text": "...", "drug": "..." } returns label + score + model\_version.

### **Dashboard (Streamlit):**

- Query aggregated tables: reviews\_by\_drug, sentiment\_trends (precomputed in DB).
  - Features: search reviews by drug, filter by dates, sentiment distribution, top phrases (wordcloud or top n-grams), export CSV.
- 

## **8. Docker / docker-compose (basic outline)**



docker-compose.yml (conceptual)

version: '3.7'

services:

postgres:

image: postgres:14

environment:

POSTGRES\_DB: reviews

POSTGRES\_USER: user

POSTGRES\_PASSWORD: pass

volumes:

- pgdata:/var/lib/postgresql/data

airflow:

build: ./infra/airflow

depends\_on: [postgres]

environment:

-

AIRFLOW\_\_CORE\_\_SQL\_ALCHEMY\_CONN=postgresql+psycopg2://user:pass@postgres:5432/reviews

mlflow:

image: mlflow

ports: ["5000:5000"]

streamlit:

build: ./src/dashboard

ports: ["8501:8501"]

redis: # optional for celery executor

image: redis:6

volumes:

pgdata:

Run:

docker-compose up --build

---

## 9. CI/CD & Tests

- **Unit tests:** pytest for preprocessing, db client, model utils.
- **Integration tests:** small end-to-end with sample dataset using a local sqlite or test Postgres.
- **GitHub Actions:**
  - Lint (flake8), tests, build docker images, push to registry (DockerHub/GHCR).
  - Optional: On push to main, trigger a deploy workflow (if you have infra).

Sample GH action job names: lint, test, build\_and\_push.

---

## 10. Security / Compliance / Pharma-specific considerations

Pharma space is sensitive — follow these guidelines in doc docs/compliance.md:

- **PII / PHI:** Identify and remove Protected Health Information in reviews. Use deterministic redaction (regex + SpaCy patterns). Keep original raw data encrypted and access-controlled.
- **Encryption:** TLS for all endpoints; at-rest encryption for DB & object storage (Postgres + minio).
- **Access control:** RBAC for Airflow and MLflow. Use strong secrets management (do **not** store secrets in repo).
- **Audit logs:** Keep thorough logging of data access, Airflow runs, model deployments, and predictions (why, when, by whom).
- **Data retention & consent:** If scraping third-party sites, ensure terms-of-service compliance; respect robots.txt and rate limits; include a retention policy (e.g., delete raw scraped PII after X months).
- **Reproducibility:** Pin versions in requirements.txt & Dockerfile, record random seeds in MLflow.

---

## 11. Monitoring & Observability

- **Model health metrics:** prediction distribution drift, input length, NA rates.
  - **Alerting:** Airflow task failures, model degradation (sudden drop in confidence), service downtime.
  - **Tools:**
    - Simple: structured logs with ELK stack (Elastic, Logstash, Kibana) or just CloudWatch/Stackdriver in cloud.
    - Advanced: Prometheus + Grafana for metrics; integrate with MLflow metrics.
- 

## 12. Example code snippets

### Preprocess (sketch)

```
# src/etl/preprocess.py (sketch)
```

```
import re, spacy
```

```
nlp = spacy.load("en_core_web_sm")
```

```
DOSAGE_RE = re.compile(r'\b\d+(\.\d+)?\s*(mg|ml|g|mcg)\b', re.IGNORECASE)
```

```
def clean_text(text):
```

```
    text = text or ""
```

```
    text = re.sub(r'<.*?>', '', text)
```

```
    text = DOSAGE_RE.sub('<DOSAGE>', text)
```

```
    text = text.replace('\n', ' ')
```

```
    return text.strip()
```

```
def lemmatize(text):
```

```
    doc = nlp(text)
```

```
lemmas = [tok.lemma_ for tok in doc if not tok.is_stop and not tok.is_punct]

return " ".join(lemmas)
```

### **FastText train (sketch)**

```
# src/models/train_fasttext.py

import fasttext

# fastText expects train file with: __label__POSITIVE text...

model = fasttext.train_supervised('train.txt', lr=0.1, epoch=10, wordNgrams=2, dim=100)

model.save_model('artifacts/fasttext_model.bin')
```

### **Airflow task (sketch)**

```
# airflow_dags/drug_reviews_dag.py

from airflow import DAG

from airflow.operators.python import PythonOperator

from datetime import datetime


def scrape_task(**ctx): ...

def preprocess_task(**ctx): ...

def train_task(**ctx): ...


with DAG('drug_reviews', start_date=datetime(2025,1,1), schedule_interval='@daily') as dag:

    t1 = PythonOperator(task_id='scrape', python_callable=scrape_task)

    t2 = PythonOperator(task_id='preprocess', python_callable=preprocess_task)

    t3 = PythonOperator(task_id='train', python_callable=train_task)

    t1 >> t2 >> t3
```

---

## **13. LLD Sequence / Data Flow (concise)**

1. Scrapy spider scrapes pages → pushes JSON to file / Kafka (optional) → Airflow scrape\_reviews task loads into raw\_reviews.
  2. validate\_raw checks & dedups.
  3. clean\_and\_preprocess runs SpaCy/NLTK transforms, writes processed\_reviews.
  4. vectorize\_features transforms to embeddings/features, stores artifacts.
  5. train\_or\_update\_model runs training, logs to MLflow.
  6. batch\_infer\_new\_reviews annotates sentiments in DB.
  7. update\_dashboard\_data writes aggregated tables for Streamlit.
  8. Users interact with Streamlit which reads DB or calls prediction endpoint for live demo.
- 

#### 14. Enhanced features & futures to add (prioritized)

1. **NER for ADR detection** (use BioBERT or SpaCy annotated models) — high value for pharma.
  2. **Semi-supervised labeling**: Use model predictions + human-in-the-loop to build more labeled data.
  3. **Active learning**: sample uncertain predictions for manual labeling.
  4. **Drift detection**: monitor input distribution & retrain automatically.
  5. **A/B model routing**: compare FastText vs TextCNN in production.
- 

#### 15. Next practical steps (what you can do right now)

1. Create repo from folder structure above.
2. Implement a minimal Scrapy spider to pull sample reviews or download the UCI Drug Review dataset.
3. Stand up Postgres + Airflow via docker-compose and run a simple DAG that inserts sample data into raw\_reviews.
4. Build preprocessing functions (clean\_text, lemmatize) and test locally.
5. Train a FastText baseline with sample labeled data and log to MLflow.

6. Make a simple Streamlit app that reads sample results.

---

If you want, I can **generate** any of the following right now (pick one or I'll choose a useful next step):

- A complete **Airflow DAG file** (drug\_reviews\_dag.py) tailored to this stack (with real task code stubs).
- A ready-to-run **docker-compose.yml** that spins up Postgres + Airflow + MLflow + Streamlit for local dev.
- The **Postgres init.sql schema** and minimal Scrapy spider example.
- A **FastText training script** with sample train.txt generator from processed\_reviews.

Tell me which concrete artifact you'd like me to produce first and I'll generate it immediately.